XSD Schema to Table Converter (`xsd2table.py`)

This Python script efficiently parses a directory of XML Schema Definition (XSD) files and generates a summary table in both CSV and Excel formats. The primary goal is to provide a consolidated view of all elements and attributes across multiple schemas, clearly indicating whether each item is mandatory (`MAND`), optional (`OPT`), or not present in a given schema.

Name	Type	appliccrossreftable	brdoc	brex	checklist	comment	comrep
Description	Element	0	0	0	0	0	0
SITA	Element	MAND	MAND	MAND	MAND	MAND	MAND
accessDoorOrPanel	Element	0	0	0	0	0	0
accessFrom	Element	0	0	0	0	0	MAND
accessPoint	Element	0	0	0	0	0	MAND
accessPointAlts	Element	0	0	0	0	0	MAND
accessPointIdent	Element	0	0	0	0	0	MAND
accessPointRef	Element	0	MAND	MAND	MAND	MAND	MAND
accessPointRefGroup	Element	0	0	0	0	0	MAND
accessPointRepository	Element	0	0	0	0	0	MAND
accessPointSpec	Element	0	0	0	0	0	MAND

Features

- **Comprehensive Summary:** Aggregates element and attribute definitions from all `.xsd` files in a specified folder.
- **Clarity on Requirements:** Determines if an item is mandatory (`minOccurs` != "0" for elements, `use` = "required" for attributes) or optional.
- **Dual Output:** Generates both a plain-text `xsd_summary.csv` file and a formatted `xsd_summary.xlsx` Excel file for easy analysis.
- **Performance:** Utilizes multithreading to parse multiple XSD files concurrently, significantly speeding up execution for large schema sets.
- **Robust Logging:** Creates a detailed log file (`logs/xsd_parser.log`) to trace the script's execution, record progress, and capture any errors for easy troubleshooting.
- **Debugging Mode:** Includes a `--debug` flag to enable verbose logging for in-depth analysis.
- **Dependency Checks:** Verifies that all required third-party libraries are installed before running.

Prerequisites

- Python 3.6+
- Required Python libraries. You can install them using pip:
  Bash

```
None

pip install lxml openpyxl
```

-

How to Run

Execute the script from your terminal, providing the path to the folder containing your XSD files.

Basic Usage:

Bash

```
None
python xsd2table.py /path/to/your/xsd_folder
```

Example:

If your schemas are in a subfolder named `xml_schema_flat`, you would run:

Bash

```
None
python xsd2table.py xml_schema_flat
```

Command-Line Options:

- `schema_folder`: (Required) The path to the folder containing the `.xsd` files.
- `-d`, `--debug`: (Optional) Enables verbose debug-level logging to the console and log file.
- `--no-csv`: (Optional) Skips the generation of the CSV output file.
- `--no-excel`: (Optional) Skips the generation of the Excel output file.

Output

The script will produce the following files in the directory where it is run:

1. `xsd_summary.csv`: A comma-separated values file containing the summary table.
2. `xsd_summary.xlsx`: A formatted Excel spreadsheet with the same summary data.

3. `logs/xsd_parser.log`: A log file containing detailed information about the script's execution. A `logs` directory will be created if it doesn't exist.

```
# -*- coding: utf-8 -*-
"""
Parses a directory of XSD schemas to generate a summary table (CSV and
Excel)
indicating whether elements and attributes are mandatory or optional
across all schemas.
"""

# --- Version History ---
# v1.0.0 (2025-09-08): Initial version with basic parsing and CSV/Excel
output.
# v2.0.0 (2025-09-08):
#   - Added robust dependency checking, multithreading, and enhanced
logging.
# v2.0.1 (2025-09-08): Reverted default value for non-existent items from
'' to '0'.
# v2.1.0 (2025-09-08):
#   - Corrected element parsing logic to find both definitions (`name`)
and references (`ref`).
#   - Implemented "OPT-wins" logic to correctly identify optional
elements.

import os
import sys
import argparse
import logging
import csv
from concurrent.futures import ThreadPoolExecutor, as_completed

# --- Dependency Check ---
try:
    from lxml import etree
except ImportError:
```

```
    print("Error: 'lxml' library not found. Please install it using 'pip
install lxml'")
    sys.exit(1)

try:
    from openpyxl import Workbook
    from openpyxl.styles import Font, PatternFill
    from openpyxl.utils import get_column_letter
except ImportError:
    Workbook = None

# --- Configuration ---
LOGS_DIR = 'logs'
LOG_FILE = os.path.join(LOGS_DIR, 'xsd_parser.log')
OUTPUT_CSV_FILE = 'xsd_summary.csv'
OUTPUT_EXCEL_FILE = 'xsd_summary.xlsx'
MAX_WORKERS = os.cpu_count() or 4

# Define XML Schema Namespace
XS_NAMESPACE = "http://www.w3.org/2001/XMLSchema"

# --- Logging Setup ---
def setup_logging(debug_mode=False):
    """Configures the logging system to output to file and console."""
    level = logging.DEBUG if debug_mode else logging.INFO
    os.makedirs(LOGS_DIR, exist_ok=True)
    logger = logging.getLogger(__name__)
    logger.setLevel(logging.DEBUG)
    if logger.hasHandlers():
        logger.handlers.clear()

    file_handler = logging.FileHandler(LOG_FILE, mode='w')
    file_handler.setLevel(logging.DEBUG)
    file_formatter = logging.Formatter('%(asctime)s - %(threadName)s -
%(levelname)s - %(message)s')
    file_handler.setFormatter(file_formatter)
    logger.addHandler(file_handler)

    console_handler = logging.StreamHandler()
    console_handler.setLevel(level)
```

```
    console_formatter = logging.Formatter('%(levelname)s: %(message)s')
    console_handler.setFormatter(console_formatter)
    logger.addHandler(console_handler)

    return logger

logger = setup_logging()

# --- XSD Parsing Logic ---

def get_element_status(element_node):
    """Determines the status (MAND/OPT) of an element based on
minOccurs."""
    min_occurs = element_node.get('minOccurs', '1')
    return 'OPT' if min_occurs == '0' else 'MAND'

def get_attribute_status(attribute_node):
    """Determines the status (MAND/OPT) of an attribute based on 'use'."""
    use = attribute_node.get('use', 'optional')
    return 'MAND' if use == 'required' else 'OPT'

def parse_xsd_file(filepath):
    """
    Parses a single XSD file to extract elements and attributes.
    This version correctly handles both element definitions and
references.
    """
    schema_name = os.path.splitext(os.path.basename(filepath))[0]
    elements_info = {}
    attributes_info = {}
    logger.info(f"Processing schema: {schema_name}")

    try:
        tree = etree.parse(filepath)
        root = tree.getroot()
        namespaces = {'xs': XS_NAMESPACE}

        # CORRECTED: Find xs:element tags with either a 'name' or a 'ref'
attribute.
```

```
        for element_node in root.xpath('.//xs:element[@name or @ref]',
namespaces=namespaces):
            # The name can be from the 'name' attribute (definition) or
'ref' attribute (reference).
            name = element_node.get('name') or element_node.get('ref')
            if not name:
                continue

            status = get_element_status(element_node)

            # "OPT-wins" LOGIC: If an element is ever found to be
optional, it stays optional.
            current_status = elements_info.get(name)
            if current_status != 'OPT':
                elements_info[name] = status
                if status == 'OPT':
                     logger.debug(f"Schema '{schema_name}': Element
'{name}' marked as OPT.")

        # Attribute logic remains the same as 'ref' is less common and
doesn't affect 'use'.
        for attribute_node in root.xpath('.//xs:attribute[@name]',
namespaces=namespaces):
            name = attribute_node.get('name')
            status = get_attribute_status(attribute_node)
            attributes_info[name] = status

        logger.info(f"Successfully processed schema: {schema_name}")
        return schema_name, elements_info, attributes_info

    except etree.XMLSyntaxError as e:
        logger.error(f"XML Syntax Error parsing {filepath} (line
{e.lineno}): {e}")
    except Exception as e:
        logger.error(f"An unexpected error occurred while parsing
{filepath}: {e}")

    return schema_name, None, None

def process_xsd_folder(folder_path):
```

```
    """Processes all XSD files in a folder using a thread pool."""
    all_elements_data = {}
    all_attributes_data = {}
    schema_names = []

    xsd_files = [os.path.join(folder_path, f) for f in
os.listdir(folder_path) if f.endswith('.xsd')]

    if not xsd_files:
        logger.warning(f"No XSD files found in '{folder_path}'.")
        return {}, {}, []

    logger.info(f"Found {len(xsd_files)} XSD files. Starting parallel
processing with {MAX_WORKERS} workers.")

    with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
        future_to_xsd = {executor.submit(parse_xsd_file, xsd): xsd for xsd
in xsd_files}
        for future in as_completed(future_to_xsd):
            try:
                schema_name, elements, attributes = future.result()
                if elements is None or attributes is None:
                    continue
                schema_names.append(schema_name)
                for elem_name, status in elements.items():
                    all_elements_data.setdefault(elem_name,
{})[schema_name] = status
                for attr_name, status in attributes.items():
                    all_attributes_data.setdefault(attr_name,
{})[schema_name] = status
            except Exception as e:
                logger.error(f"An error occurred fetching result for a
schema: {e}")

    return all_elements_data, all_attributes_data, sorted(schema_names)

# --- Output Generation ---

def generate_csv_output(header, element_rows, attribute_rows,
output_file):
```

```
    """Writes the collected data to a CSV file."""
    logger.info("Generating CSV output...")
    try:
        with open(output_file, 'w', newline='', encoding='utf-8') as f:
            writer = csv.writer(f)
            writer.writerow(header)
            writer.writerows(element_rows)
            if element_rows and attribute_rows:
                writer.writerow([])
            writer.writerows(attribute_rows)
        logger.info(f"Successfully generated CSV file: {output_file}")
    except IOError as e:
        logger.error(f"Error writing to CSV file {output_file}: {e}")

def generate_excel_output(header, element_rows, attribute_rows,
output_file):
    """Writes the collected data to a formatted Excel file."""
    if Workbook is None:
        logger.error("Cannot generate Excel file: 'openpyxl' is not
installed. Run 'pip install openpyxl'")
        return

    logger.info("Generating Excel output...")
    try:
        wb = Workbook()
        ws = wb.active
        ws.title = "XSD Summary"
        header_font = Font(bold=True)
        header_fill = PatternFill(start_color="D9D9D9",
end_color="D9D9D9", fill_type="solid")
        ws.append(header)
        for cell in ws[1]:
            cell.font = header_font
            cell.fill = header_fill
        for row in element_rows:
            ws.append(row)
        if element_rows and attribute_rows:
            ws.append([])
        for row in attribute_rows:
            ws.append(row)
```

```
            for col_idx, column_cells in enumerate(ws.columns, 1):
                max_length = 0
                column_letter = get_column_letter(col_idx)
                for cell in column_cells:
                    if cell.value:
                        max_length = max(max_length, len(str(cell.value)))
                adjusted_width = max_length + 2
                ws.column_dimensions[column_letter].width = adjusted_width
            wb.save(output_file)
            logger.info(f"Successfully generated Excel file: {output_file}")
        except Exception as e:
            logger.error(f"An unexpected error occurred during Excel
generation: {e}")

# --- Main Execution ---

def main():
    """Main function to parse arguments and orchestrate the script
execution."""
    parser = argparse.ArgumentParser(
        description="Parse XSD schemas to create a summary of element and
attribute requirements.",
        epilog="Example: python xsd2table.py ./my_schemas --debug"
    )
    parser.add_argument("schema_folder", help="Path to the folder
containing XSD schema files.")
    parser.add_argument("-d", "--debug", action="store_true", help="Enable
debug logging.")
    parser.add_argument("--no-csv", action="store_true", help="Do not
generate the CSV output file.")
    parser.add_argument("--no-excel", action="store_true", help="Do not
generate the Excel output file.")
    args = parser.parse_args()

    global logger
    logger = setup_logging(debug_mode=args.debug)

    schema_folder = args.schema_folder
    if not os.path.isdir(schema_folder):
```

```
        logger.error(f"Input path is not a valid directory:
{schema_folder}")
        return

    logger.info(f"Starting XSD processing for folder: {schema_folder}")
    all_elements_data, all_attributes_data, schema_names =
process_xsd_folder(schema_folder)

    if not schema_names:
        logger.warning("No schemas were successfully processed. Exiting.")
        return

    header = ["Name", "Type"] + schema_names
    element_rows, attribute_rows = [], []

    for elem_name in sorted(all_elements_data.keys()):
        row = [elem_name, "Element"]
        for schema in schema_names:
            status = all_elements_data[elem_name].get(schema, '0')
            row.append(status)
        element_rows.append(row)

    for attr_name in sorted(all_attributes_data.keys()):
        row = [attr_name, "Attribute"]
        for schema in schema_names:
            status = all_attributes_data[attr_name].get(schema, '0')
            row.append(status)
        attribute_rows.append(row)

    if not args.no_csv:
        generate_csv_output(header, element_rows, attribute_rows,
OUTPUT_CSV_FILE)
    if not args.no_excel:
        generate_excel_output(header, element_rows, attribute_rows,
OUTPUT_EXCEL_FILE)

    logger.info("Script execution finished successfully.")

if __name__ == "__main__":
    main()
```